

- See previous practice problem sets for instructions.
- Treat all arrays as 0-indexed. You may assume the following in your analyses:
 - Storing an integer takes constant space, irrespective of how large the integer is. This is clearly not true (even in theory), but our focus in these problems is not on the intricacies of dealing with large integers. So we make this assumption for the sake of simplifying the analysis.
 - Each operation—arithmetic, comparison, etc.—between a pair of integers takes constant time, irrespective of how large the integers are.
 - Accessing a single array element takes constant time, irrespective of the length of the array.
- Clearly state any other assumption that you need.

1. Refer to the sorting algorithm in your solution to Problem 8 in Problem Set 2.
 - (a) Obtain a good asymptotic upper bound (that is, of the form $T(n) = \mathcal{O}(f(n))$, for some function f) on the *worst-case* running time of this algorithm on an input list of size n .
 - (b) Obtain a good asymptotic upper bound on the *best-case* running time of this algorithm on an input list of size n .
2. Read up on what a **palindrome** is, if required. Your solution to parts (c) and (d) must consist of deriving recurrence relations and solving them using the “estimate and verify” method that we saw in class.
 - (a) Write the pseudocode for a **recursive** function `IsPALINDROME(string)` that checks if the given string is a palindrome. (*Hint*: You already did something like this for Problem Set 1.)
 - (b) Prove using induction that your pseudocode of part (a) is correct. (*Hint*: Ditto.)
 - (c) Obtain a good asymptotic upper bound on the worst-case running time of your algorithm of part (a).
 - (d) Obtain a good asymptotic upper bound on the best-case running time of your algorithm of part (a).
3. Refer to your algorithm `FINDLARGEST` from Problem Set 1. For each part below, your analysis must consist of deriving recurrence relations and solving them using the “estimate and verify” method that we saw in class.
 - (a) Obtain a good asymptotic upper bound on the worst-case running time of your algorithm, in terms of the number of numbers in the input.
 - (b) Obtain a good asymptotic upper bound on the best-case running time of your algorithm, in terms of the number of numbers in the input.

4. Read up the definition of the **Longest Common Substring** problem.
 - (a) Write the pseudocode for an algorithm that solves this problem for two input strings x, y , starting with a *recursive* solution for the following simpler version: Given (x, y, i, j, r) as input where i, j, r are integers, check whether the r -length substrings of x, y that start at indices i, j , respectively, are identical. That is, whether $x[i]x[i+1] \cdots x[i+r-1]$ is identical to $y[j]y[j+1] \cdots y[j+r-1]$.
 - (b) Let m, n be the lengths of inputs x, y , respectively. Analyze your algorithm of part (a) to obtain a good asymptotic upper bound on the time it takes to solve Longest Common Substring, in terms of m and n .

5. Prove the following statement using the definition of the $\mathcal{O}()$ notation that we saw in class:

Claim

Let $f(n), g(n)$, and $h(n)$ be functions from non-negative integers to real numbers. If $f(n) = \mathcal{O}(g(n))$ and $g(n) = \mathcal{O}(h(n))$, then $f(n) = \mathcal{O}(h(n))$.

6. Prove or disprove each of the following asymptotic equations. A proof should show the existence of constants c, n_0 that satisfy the definition of $\mathcal{O}()$ which we saw in class. A disproof should show that no such constants can exist.

- (a) $3n = \mathcal{O}(2n)$
- (b) $\log_2 3n = \mathcal{O}(\log_2 2n)$
- (c) $2^{3n} = \mathcal{O}(2^{2n})$
- (d) $n^{\log_2 \log_2 n} = \mathcal{O}(2^{\sqrt{\log_2 n}})$
- (e) $2^{\sqrt{\log_2 n}} = \mathcal{O}(n^{\log_2 \log_2 n})$

7. For each of the following recurrences, unroll the recurrence to come up with an estimate $f(n)$ that satisfies $T(n) = \mathcal{O}(f(n))$. In each case, verify your estimate by induction.

You may assume bounds of the form $T(n') \leq c'$ where n', c' are fixed constants of your choice. That is, you may assume constant upper bounds for inputs of up to some constant size.

- (a) $T(n) = T(n/2) + c$, where c is a fixed positive integer.
- (b) $T(n) = T(n/2) + 3 \log_2 n$
- (c) $T(n) = 2T(n/2) + 5n\sqrt{n}$
- (d) $T(n) = 2T(n/2) + \frac{n}{\log_2 n}$
- (e) $T(n) = 4T(n/2) + an$
- (f) $T(n) = 3T(n/2) + \mathcal{O}(n)$
- (g) $T(n) = 2T(n/2) + \mathcal{O}(n)$